Library Management System

Course Name: Advanced Programming (OOPS)

Student Name: Rohil Kumar De

Roll Number: 85

Semester: Spring 2026

Problem Statement

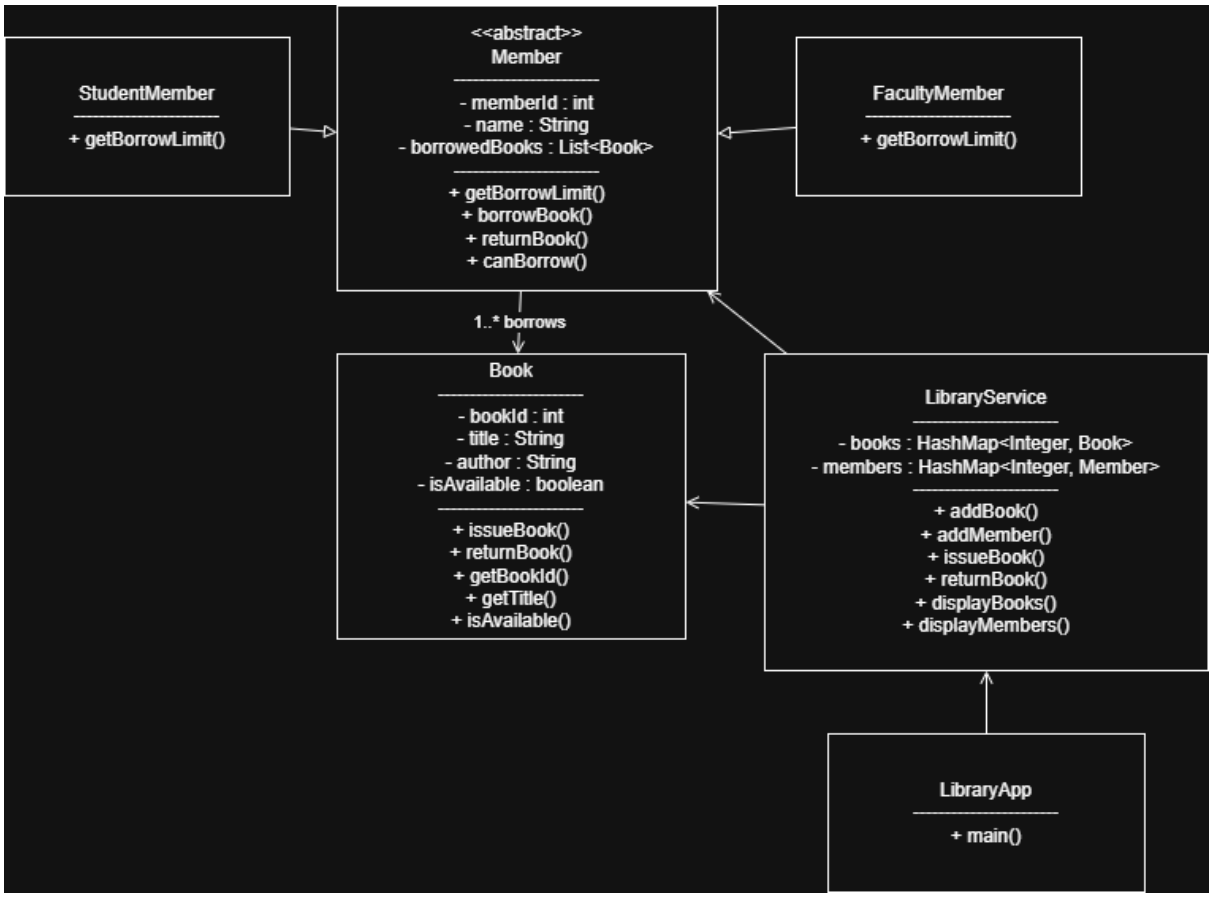
Libraries are essential components of educational institutions, providing access to books and academic resources. However, many libraries still rely on manual or semi-automated systems to manage their operations. These traditional systems often lead to inefficiencies such as difficulty in tracking issued and returned books, maintaining member records, and enforcing borrowing limits.

Manual record keeping is prone to human errors, data inconsistency, and delays in processing transactions. As the number of books and users increases, managing these records becomes increasingly complex and unreliable.

To address these challenges, there is a need for a structured and automated system that can efficiently manage library operations. The proposed Library Management System aims to provide a digital solution using Java and Object-Oriented Programming concepts. The system automates tasks such as adding books, registering members, issuing and returning books, and maintaining accurate records.

System Design

UML Class Diagram:



Description of Major Classes

Book Class

Represents a book in the library. It contains attributes such as book ID, title, author, and availability status. It provides methods to issue and return books.

Member Class (Abstract)

Defines common properties for all members such as member ID, name, and borrowed books. It includes an abstract method getBorrowLimit().

StudentMember Class

Inherits from Member and defines a borrowing limit of 3 books.

FacultyMember Class

Inherits from Member and defines a borrowing limit of 5 books.

LibraryService Class

Handles all business logic such as adding books, registering members, issuing books, returning books, and displaying records.

LibraryApp Class

Acts as the entry point of the application. It provides a menu-driven interface for user interaction.

Package Structure

The project follows a modular package structure:

- librarymanagement.model
- librarymanagement.service
- librarymanagement.exception
- librarymanagement.main

This structure ensures separation of concerns and improves maintainability.

Interaction Flow

- User selects an option from the menu
- LibraryApp receives input
- Request is passed to LibraryService
- Service validates and processes request
- Model classes are updated
- Output is displayed to user

OOP Concepts Demonstrated

Abstraction

Where used: Member abstract class

Why used: To define common structure for all members

Code Snippet:

```
public abstract class Member {
   public abstract int getBorrowLimit();
}
```

Explanation: Forces subclasses to implement their own borrowing limit.

Inheritance

Where used: StudentMember, FacultyMember

Why used: To reuse common properties of Member

Code Snippet

```
public class StudentMember extends Member {
}
```

Explanation: Avoids code duplication.

Polymorphism

Where used: getBorrowLimit() method

Code Snippet

```
@Override
public int getBorrowLimit() {
   return 3;
}
```

Explanation: Same method behaves differently based on object type

Encapsulation

Where used: Book class

```
private int bookId;
```

Explanation: Data is protected and accessed via methods.

Exception Handling

Where used: Custom exceptions

Code Snippet

```
if(book == null)
   throw new BookNotFoundException("Book not found");
```

Explanation: Prevents program crashes.

<u>Implementation Highlights</u>

Snippet 1: Book Issuing Logic

```
public void issueBook(int bookId, int memberId)
      throws BookNotFoundException,
```

```
            MemberNotFoundException,
            BookUnavailableException {
    Book = books.get(bookId);
    Member = members.get(memberId);
    if (book == null)
        throw new BookNotFoundException("Book not found.");
   if (member == null)
        throw new MemberNotFoundException("Member not found.");
   if (!book.isAvailable())
        throw new BookUnavailableException("Book already issued.");
   if (!member.canBorrow())
        throw new BookUnavailableException("Borrow limit reached.");
   book.issueBook();
    member.borrowBook(book);
}
```

Explanation:

This method handles the core functionality of issuing a book. It first retrieves the book and member using HashMap for efficient lookup. It then performs validation checks such as verifying the existence of the book and member, checking book availability, and ensuring that the member has not exceeded their borrowing limit. If all conditions are satisfied, the book is issued and added to the member's borrowed list. This ensures data consistency and prevents invalid operations.

Snippet 2: Use of Collections (HashMap & ArrayList)

```
private Map<Integer, Book> books = new HashMap<>();
private Map<Integer, Member> members = new HashMap<>();
protected List<Book> borrowedBooks = new ArrayList<>();
```

Explanation:

This snippet shows the use of Java Collections for efficient data management. HashMap is used to store books and members using their IDs as keys, allowing fast access and retrieval in constant time. ArrayList is used to maintain a list of books borrowed by each member. These data structures improve the performance and scalability of the system.

Snippet 3: Exception Handling

```
if (book == null)
   throw new BookNotFoundException("Book not found.");
if (!book.isAvailable())
   throw new BookUnavailableException("Book already issued.");
```

Explanation:

Custom exception handling is used to manage error scenarios effectively. Instead of allowing the program to crash, meaningful exceptions are thrown when invalid conditions occur, such as a missing book or issuing an unavailable book. This improves system robustness and provides clear feedback to the user.

Testing & Error Handling

Test Cases

- Issue a valid book
- Return a book
- Issue an already issued book
- Exceed borrowing limit
- Invalid book/member ID

Edge Cases

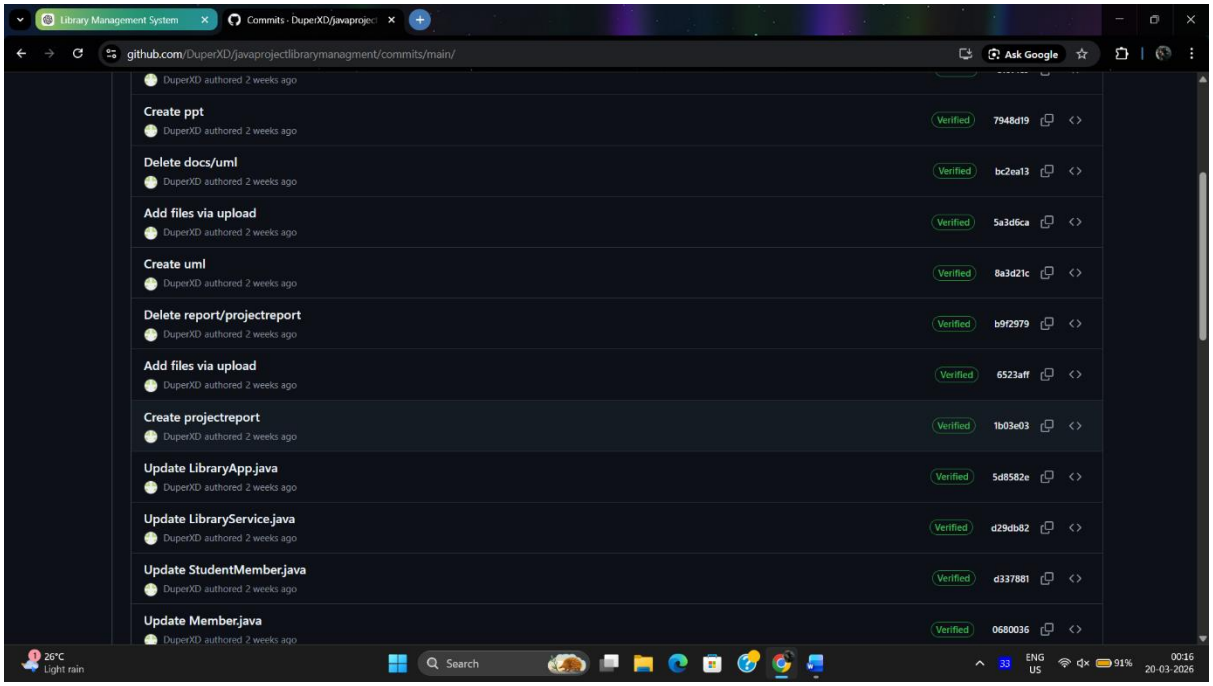
- No books in system
- No members registered
- Duplicate IDs

Failure Handling

Custom exceptions are used to handle errors gracefully. The system displays meaningful messages instead of crashing.

Git Workflow

Commit History:



- Initial project setup
- Added model classes
- Implemented service layer
- Added exception handling
- Implemented menu system

Conclusion & Future Scope

The Library Management System successfully demonstrates the implementation of Object-Oriented Programming concepts using Java. The system provides an efficient way to manage books and members, ensuring data accuracy and ease of use.

Future Scope

- Add file/database storage
- Implement GUI
- Add search functionality
- Add fine calculation system

Appendix

00:20

PCCCS495 Proposal Review – APPROVED_MINOR

S

soumadip.biswas@ieminternat... 2 Mar

To You

Dear Rohil Kumar De,

Project: library management system
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal presents a clear, relevant problem statement with a well-defined target audience. The proposed features are comprehensive and appropriate for a university-level Java OOP project, covering essential functionalities like CRUD operations, validation, and business rule enforcement. The explicit mention of core OOP concepts (Abstraction, Inheritance, Polymorphism, Encapsulation) alongside

Reply

The described layered architecture (Model,